

FOUNDATIONAL ANALYTICS PRACTICUM

E-Commerce Customer Behavior & Delivery Analytics

Unit Economics, Logistics Bottlenecks & Discount Elasticity at SwiftKart

Core Stack: Python • NumPy • Pandas • Matplotlib

Duration: 4–6 Guided Lab Hours

Prerequisites: Basic Python (No ML Required)

1. THE BUSINESS PROBLEM & EXECUTIVE CONTEXT

SwiftKart is a high-growth multi-category e-commerce platform serving major Tier 1, Tier 2, and Tier 3 Indian markets. While Gross Merchandise Value (GMV) surged over the last two quarters, operating profit margins have deteriorated due to three interlocking operational friction points:

PILLAR 1 • UNIT ECONOMICS

Surging Return Rates

Product returns have spiked significantly. Reverse freight expenses, restocking friction, and damaged-in-transit write-offs are severely impairing net realization across core categories.

PILLAR 2 • FULFILLMENT FRICTION

Logistics Bottlenecks

Fulfillment routes repeatedly miss published Estimated Delivery Dates (EDD). Unreliable delivery timelines directly trigger support escalations, poor ratings, and transit cancellations.

PILLAR 3 • MARGIN DILUTION

Discount Dependence

Aggressive promotional campaigns offer up to 50% markdowns. Leadership cannot distinguish whether discounts attract sticky, high-LTV repeat buyers or merely opportunistic churn risks.

The Analyst's Mission & Strategic Mandate

As a **Junior Data Analyst** within SwiftKart's Operations and Growth Intelligence unit, your objective is to ingest raw transactional data, clean messy operational fields, derive performance KPIs, segment customer behaviors using vectorized business rules (no machine learning algorithms), and construct publication-grade visualizations to advise logistics managers and marketing directors.

Enterprise Schema & Data Dictionary

The generated telemetry file (`ecommerce_orders.csv`) captures end-to-end customer order life-cycles across the following 11 operational attributes:

ATTRIBUTE NAME	DATA TYPE	BUSINESS DEFINITION	EXPECTED DOMAIN / RANGE
Order_ID	String (Object)	Unique platform transaction reference identifier	ORD1000 – ORD1999
Customer_ID	String (Object)	Unique consumer account identifier	CUST100 – CUST349 (Repeat buyers)
Category	String (Categorical)	Primary merchandise vertical	Electronics, Fashion, Home, Books, Beauty
Payment_Method	String (Categorical)	Checkout gateway or settlement mechanism	Credit Card, UPI, Debit Card, COD, upi
Order_Value	Float64	Gross order value prior to discounts (INR)	Exponential dist. (~₹200 to ₹8,000+)
Discount_Percent	Int64	Promotional discount applied at checkout	0%, 5%, 10%, 15%, 20%, 30%, 50%
Est_Delivery_Days	Int64	Committed delivery turnaround promised at checkout	2 to 7 Business Days
Actual_Delivery_Days	Int64	True transit time from warehouse dispatch to door	1 to 14 Business Days
Customer_Rating	Float64	Post-delivery CSAT feedback score	1 to 5 Stars (contains missing values)
Returned	String (Categorical)	Indicates if a return/refund ticket was logged	"Yes" (~18%), "No" (~82%)
City_Tier	String (Categorical)	Destination fulfillment urban classification	Tier 1 (45%), Tier 2 (35%), Tier 3 (20%)

2. DATASET GENERATION ENGINE

Execute the reproducible Python script below to synthesize the raw operational dataset (`ecommerce_orders.csv`). The script establishes an initial cohort of 1,000 baseline records and subsequently injects real-world anomalies, duplicate log captures, and missing data points:

```
import numpy as np
import pandas as pd

np.random.seed(42)
n = 1000

order_ids = [f"ORD{1000 + i}" for i in range(n)]
customer_ids = [f"CUST{np.random.randint(100, 350)}" for _ in range(n)]
categories = np.random.choice(["Electronics", "Fashion", "Home & Kitchen", "Books", "Beauty"],
                               size=n, p=[0.25, 0.30, 0.20, 0.10, 0.15])
payment_methods = np.random.choice(["Credit Card", "UPI", "Debit Card", "Cash on Delivery", "upi"], size=n)

order_values = np.round(np.random.exponential(scale=1200, size=n) + 200, 2)
discount_pct = np.random.choice([0, 5, 10, 15, 20, 30, 50], size=n, p=[0.2, 0.2, 0.2, 0.15, 0.1, 0.1, 0.05])
delivery_days_estimated = np.random.randint(2, 8, size=n)
delivery_days_actual = delivery_days_estimated + np.random.choice([-1, 0, 1, 2, 4, 7], size=n, p=[0.15, 0.45, 0.20, 0.10, 0.05, 0.05])
customer_rating = np.random.choice([1, 2, 3, 4, 5, np.nan], size=n, p=[0.08, 0.12, 0.25, 0.35, 0.15, 0.05])
returned = np.random.choice(["Yes", "No"], size=n, p=[0.18, 0.82])
city_tier = np.random.choice(["Tier 1", "Tier 2", "Tier 3"], size=n, p=[0.45, 0.35, 0.20])

df = pd.DataFrame({
    "Order_ID": order_ids, "Customer_ID": customer_ids, "Category": categories,
    "Payment_Method": payment_methods, "Order_Value": order_values,
    "Discount_Percent": discount_pct, "Est_Delivery_Days": delivery_days_estimated,
    "Actual_Delivery_Days": delivery_days_actual, "Customer_Rating": customer_rating,
    "Returned": returned, "City_Tier": city_tier
})

# Inject real-world data issues: 15 exact duplicate records and 20 missing order values
df = pd.concat([df, df.iloc[:15]], ignore_index=True)
df.loc[np.random.choice(df.index, 20), "Order_Value"] = np.nan
df.to_csv("ecommerce_orders.csv", index=False)
print("Generated 'ecommerce_orders.csv' with", len(df), "records.")
```

3. DETAILED STUDENT TASKS & PROBLEM SETS

Section A: Data Inspection & Cleaning (Pandas)

Weight: 20 Marks

- A1. Load and Validate:** Import `ecommerce_orders.csv` using Pandas. Print dataframe dimensions (`df.shape`), verify column types (`df.dtypes`), and compute total null/missing values per column (`df.isnull().sum()`).
- A2. Primary Key Deduplication:** Identify duplicate rows on `Order_ID`. Drop redundant entries while keeping the first occurrence (`keep='first'`). Confirm that the dataframe count returns to exactly 1,000 records.
- A3. String Standardization:** Clean the `Payment_Method` column: strip leading/trailing whitespace and normalize casing so that `'upi'` harmonizes cleanly to `'UPI'` and payment types follow Title Case conventions.
- A4. Category-Wise Median Imputation:** Impute missing `Order_Value` entries using the **median order value of that respective product Category** via `df.groupby('Category')['Order_Value'].transform('median')`. *Explain why category-level median imputation is superior to global mean imputation for skewed spending data.*
- A5. Rating Handling & Justification:** Fill missing `Customer_Rating` values with either the overall mode or a sentinel placeholder (`-1`). Provide an explicit analytical justification in your documentation defending your choice.

Section B: Numerical Operations & Vectorized Logic (NumPy)**Weight: 20 Marks**

- **B1. Vectorized Delay Metric:** Convert `Est_Delivery_Days` and `Actual_Delivery_Days` to NumPy ndarrays. Calculate the vectorized variance array:

```
delivery_delay = actual_delivery_days - est_delivery_days
```

Assign the resulting array back to the DataFrame as `Delivery_Delay`.

- **B2. Delivery Status Classification via `np.select()`:** Construct column `Fulfillment_Status` using conditional boolean arrays and choice lists:
 - "Early" if `delivery_delay < 0`
 - "On-Time" if `delivery_delay == 0`
 - "Minor Delay" if `0 < delivery_delay ≤ 2`
 - "Severe Delay" if `delivery_delay > 2`

Requirement: No iterative Python loops (for) or row-wise `apply()` statements are permitted.

- **B3. Percentile-Based High-Value Outlier Flagging:** Use `np.percentile()` to calculate the exact 95th percentile threshold of `Order_Value`. Flag all orders strictly exceeding this cutoff as `"High-Value Order"`.
- **B4. Vectorized Net Realized Revenue:** Derive true realized commercial revenue by vectorizing discount deductions:

```
Net_Revenue = Order_Value * (1 - Discount_Percent / 100)
```

Section C: Business Analytics & Aggregations (Pandas)**Weight: 25 Marks**

- **C1. Category Performance Matrix:** Group transactions by `Category` and aggregate: (1) Total Net Revenue, (2) Average Discount Offered, and (3) Return Rate percentage (`(Returned == 'Yes').mean() * 100`). Sort results in descending order of revenue.
- **C2. Regional Fulfillment Friction Analysis:** Group data by `City_Tier` and compute the percentage of orders facing `"Severe Delay"`. Identify which city tier experiences the worst logistics failure rate and diagnose potential root causes.
- **C3. Customer Sentiment Cross-Tabulation:** Construct a Pandas pivot table (`pd.pivot_table`) displaying average `Customer_Rating` with `Category` as index rows and `Fulfillment_Status` as columns. Analyze rating drops across delay brackets.
- **C4. Rule-Based Customer Segmentation (No ML):** Segment every transaction using boolean vector filtering:
 - "High-Risk Transaction": `Discount_Percent ≥ 30% AND Returned == "Yes"`
 - "Loyal & Satisfied": `Order_Value > median(Order_Value) AND Customer_Rating ≥ 4 AND Returned == "No"`
 - "Standard Order": All remaining cases.

Section D: Visual Analytics & Communicative Storytelling (Matplotlib)

Weight: 20 Marks

- All plots must be constructed using Matplotlib directly. Charts must satisfy professional publication standards: clear titles, labeled axes with appropriate unit notations, distinct contrasting colors, and subtle gridlines (`alpha=0.3`).
- **D1. Commercial Contribution Bar Chart:** Total Net Revenue by Product Category, sorted in descending order with distinct bar colors and currency-formatted tick values.
 - **D2. Fulfillment Distribution & Benchmark Histogram:** Distribution of `Actual_Delivery_Days` with a prominent dashed vertical line showing the average estimated delivery benchmark.
 - **D3. Discount Elasticity vs. Return Propensity Scatter Plot:** Map `Discount_Percent` (X-axis) against `Order_Value` (Y-axis), with coordinate markers colored by return status (`Returned == 'Yes'` vs `'No'`).
 - **D4. Logistics Impact on Sentiment Grouped Bar Chart:** Average `Customer_Rating` compared across `Fulfillment_Status` categories to visually prove the impact of delivery latency on reviews.

4. RIGOROUS ACADEMIC GRADING RUBRIC (100 MARKS TOTAL)

Evaluation Pillar	Core Assessment Focus & Quality Criteria	Target Tool	Marks
Data Quality & Cleaning	Deduplication accuracy; whitespace & casing cleanup; category-level median imputation; sound justification of customer rating handling.	Pandas	20
NumPy Operations	Vectorized latency math; multi-condition branching via <code>np.select</code> without loops; percentile logic via <code>np.percentile</code> ; net revenue arithmetic.	NumPy	20
Pandas Aggregations	Multi-metric groupbys; multi-index pivot tables; compound boolean conditional masks for customer behavioral segmentation.	Pandas	25
Matplotlib Charts	Plot type appropriateness, scale legibility, axis formatting with units, visual contrast, annotations, and clean presentation.	Matplotlib	20
Strategic Synthesis	Formulation of 3–4 executive recommendations directly addressing return rate reduction, delivery routing, and discount targeting.	Synthesis	15
Total Cumulative Practicum Score			100 Marks

5. Project Deliverables & Submission Protocol

- **Jupyter Notebook (`swiftkart_analytics.ipynb`):** Fully documented notebook with numbered task cells matching Sections A through D, markdown annotations, executable code, and rendered figures.
- **Executive Summary Memorandum:** A 1-page business brief answering: (1) Which tier requires immediate logistics overhaul? (2) Does deep discounting yield high-risk returns? (3) Recommended discount thresholds.
- **Cleaned Dataset Export (`ecommerce_orders_cleaned.csv`):** Exported dataset containing all original fields plus newly engineered features (`Delivery_Delay` , `Fulfillment_Status` , `Net_Revenue` , `Order_Segment`).